Banking RAG Customer Support System

RAG · HITL · LangGraph · Groq · ChromaDB · Streamlit

Core Design Principle

The system integrates Retrieval-Augmented Generation with Human-in-the-Loop routing to ensure both automation and safety in handling sensitive banking queries. Every LLM response is grounded in retrieved factual context; every high-risk query is intercepted before reaching the LLM.

Document Type	Technical Documentation	**Audience**	Software Engineers, ML Engineers
Codebase	app.py + main.py	**Purpose**	Complete system explanation for engineers
Primary LLM	llama-3.1-8b-instant (Groq)	**Fallback LLM**	Falcon-RW-1B (HuggingFace)
Vector DB	ChromaDB	**Embedding**	all-MiniLM-L6-v2 (384-dim)

Table of Contents

1 Introduction — What is RAG and Why It Matters

- 1.1 RAG Fundamentals
- 1.2 Why RAG for Banking
- 1.3 Use Case Overview

2 System Architecture Explanation

- 2.1 HLD Component Interactions
- 2.2 Data Flow Walkthrough

3 Design Decisions

- 3.1 Chunk Size and Overlap Strategy
- 3.2 Embedding Model Selection
- 3.3 Retrieval Approach
- 3.4 Prompt Design Logic

4 Workflow Explanation — LangGraph

- 4.1 Node Responsibilities
- 4.2 State Transitions

5 Conditional Logic & Intent Detection

6 HITL Implementation

- 6.1 Role of Human Intervention
- 6.2 Benefits and Limitations

7 Challenges and Trade-offs

8 Testing Strategy

- 8.1 Testing Approach
- 8.2 Sample Test Queries

9 Future Enhancements

1. Introduction — What is RAG and Why It Matters

1.1 RAG Fundamentals

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances large language model (LLM) outputs by conditioning generation on factual context retrieved at query-time from an external knowledge source. Unlike a pure LLM — which generates text solely from patterns learned during pre-training — a RAG system dynamically fetches the most relevant document passages and provides them as grounding context to the LLM before generation.

Component	Role in RAG
Retriever	Given a user query, searches a vector database for semantically similar document chunks. Uses embedding similarity (cosine distance) rather than keyword matching.
Augmentation	The retrieved chunks are prepended to the prompt, providing the LLM with factual, domain-specific context it would otherwise lack or hallucinate.
Generator	The LLM receives the augmented prompt (context + question) and produces a grounded, factual answer constrained to the provided evidence.

1.2 Why RAG is Needed for Banking

Banking is a high-stakes, regulation-heavy domain where accuracy is non-negotiable. A pure LLM will hallucinate loan rates, FDIC limits, KYC requirements, and policy details. RAG resolves four fundamental problems:

- **Accuracy:** LLMs trained on internet-scale data do not know your bank's specific policies, fee structures, or product terms. RAG grounds every answer in your official documentation.
- **Freshness:** LLM knowledge cuts off at training time. RAG queries a knowledge base that can be updated independently — new PDF uploaded, database re-indexed.
- **Auditability:** Every answer traces back to specific retrieved chunks. Engineers and compliance teams can inspect which source document drove each response.
- **Hallucination Control:** Prompt constraints ('answer only from the provided context') significantly reduce confabulation — a critical requirement for regulated industries.

1.3 Use Case Overview

This system targets a **banking customer support** scenario where customers submit natural-language queries via a chat interface. The knowledge base is a curated PDF (banking_customer_support_qa.pdf) containing Q&A; pairs covering:

- Account management (opening, closing, enquiries)
- KYC compliance requirements
- Credit and debit card operations
- Transaction disputes and history
- Fraud awareness and reporting procedures

- Loan and interest rate information

Sensitive queries (fraud reports, card loss) are intercepted by the HITL layer before reaching the LLM, ensuring human oversight for high-risk situations.

2. System Architecture Explanation

2.1 HLD Component Interactions

The system comprises seven interacting components arranged in a linear pipeline with a conditional branch at the routing layer. Component boundaries are enforced by the app.py / main.py module split:

Interaction	Description
Streamlit UI → main.py	app.py calls ask_question(query). This is the only public interface of main.py — clean API boundary.
HITL Module → File System	needs_human() calls log_human_request() which writes to human_requests.txt. No return value to LLM pipeline.
Retriever → ChromaDB	retriever.invoke(query) triggers an HNSW similarity search inside the persisted Chroma collection.
Retriever → Prompt Builder	Retrieved Document objects are unpacked to page_content strings and joined into the context block.
Prompt Builder → Groq API	Assembled prompt string is wrapped in an OpenAI-compatible chat message and sent to Groq's REST endpoint.
Groq API → Streamlit	Raw answer string propagates back through ask_question() return value to app.py for rendering.
Groq Failure → Falcon Pipeline	Exception handling redirects the same prompt to the locally loaded HuggingFace pipeline.

2.2 Data Flow Walkthrough

Two distinct flows operate in this system — the one-time ingestion flow and the per-query inference flow:

Ingestion Flow (startup, runs once)

- PyPDFLoader reads banking_customer_support_qa.pdf → raw Document list (one per page).
- RecursiveCharacterTextSplitter splits each Document → ~500-char chunks with 50-char overlap.
- HuggingFaceEmbeddings.embed_documents() converts each chunk to a 384-dim float32 vector.
- Chroma.from_documents() writes all vectors + metadata to ./db (SQLite + HNSW index).
- vectorstore.as_retriever(k=3) is stored as a module-level variable for query-time use.

Query Flow (per user message)

- User input arrives at ask_question(query) from app.py.
- needs_human(query.lower()) evaluates keyword rules — returns bool.
- If True: log query → return escalation message (LLM bypassed entirely).
- If False: retriever.invoke(query) → list of 3 Document objects from ChromaDB.
- Guard check: if not docs → return 'I don\'t know.' (prevents empty-context generation).
- context = '\n\n'.join([doc.page_content for doc in docs]) builds the context string.

- Prompt template assembled: system instruction + context block + question + 'Answer:' suffix.
- groq_client.chat.completions.create() called → response.choices[0].message.content.strip().
- On any exception: fallback_llm(prompt)[0]['generated_text'].replace(prompt,'').strip().
- Answer string returned to app.py → rendered with st.success() or st.error().

3. Design Decisions

3.1 Chunk Size and Overlap Strategy

Chunking is the most consequential parameter in a RAG system. The chosen values balance retrieval precision against context completeness:

chunk_size = 500

500 characters (~80–120 words) fits well within MiniLM-L6-v2's 256-token attention window. Larger chunks risk truncation by the embedding model, causing loss of tail content. Smaller chunks (< 200 chars) are too granular — individual sentences lack sufficient context for semantic search.

chunk_overlap = 50

50-character overlap (10% of chunk size) ensures that answers spanning a chunk boundary are not lost. Without overlap, a sentence split across chunks might not be retrieved by either chunk's embedding. The overlap duplicates boundary content, making both chunks retrievable.

Splitter Hierarchy

RecursiveCharacterTextSplitter tries \n\n → \n → ' ' → '' in order. This respects paragraph and sentence boundaries first, resorting to character splits only as a last resort — preserving natural reading units.

3.2 Embedding Model Selection

all-MiniLM-L6-v2 was selected from the Sentence Transformers catalogue based on four criteria:

Criterion	Justification
Semantic Quality	Fine-tuned on 1B+ sentence pairs using contrastive learning — outperforms bag-of-words and TF-IDF on Q&A; retrieval benchmarks (BEIR).
Speed	6-layer transformer (MiniLM compression) runs at ~14,000 sentences/sec on CPU — suitable for real-time embedding.
Vector Dimension	384-dim vectors are compact: low memory footprint in ChromaDB, fast cosine computation.
No API Required	Runs entirely locally — no third-party embedding API, no PII leakage, no rate limits.

3.3 Retrieval Approach

The retriever uses **dense vector similarity search** (cosine distance via HNSW index) rather than sparse keyword search (BM25/TF-IDF). This choice enables semantic matching — a query about 'fraudulent charges' retrieves chunks about 'unauthorised transactions' even without exact keyword overlap. Key retrieval parameters:

- **k = 3:** Returns the 3 most relevant chunks. Empirically, k=3 provides sufficient context for most banking Q&A; without bloating the prompt beyond the LLM's optimal attention range.

- **No Score Threshold:** All top-k chunks are returned regardless of similarity score. A score threshold would risk returning 0 results for novel queries — the 'I don\'t know' guard handles truly empty retrievals.
- **Cosine Similarity:** Normalised distance measure invariant to vector magnitude — appropriate for MiniLM's L2-normalised output embeddings.

3.4 Prompt Design Logic

Prompt engineering directly controls LLM behaviour. The prompt template encodes four constraints:

```
prompt = f'''
You are a banking customer support assistant.
Answer the question clearly in 2-3 lines.
DO NOT generate multiple questions.
DO NOT repeat the context.
Context:
{context}
Question:
{query}
Answer:
'''
```

Element	Design Rationale
Role Instruction	'You are a banking customer support assistant' — sets domain persona, constrains off-topic generation.
Length Constraint	'Answer clearly in 2-3 lines' — prevents verbose, rambling answers unsuitable for a chat interface.
Anti-Hallucination	'DO NOT repeat the context' — stops the model from parroting retrieved text verbatim, forcing synthesis.
Anti-Question-Gen	'DO NOT generate multiple questions' — prevents the model from asking clarifying questions that cannot be answered.
'Answer:' Suffix	Elicits completion-style generation — model fills in after the prefix rather than generating a conversational preamble.

4. Workflow Explanation — LangGraph

The current implementation uses LangChain LCEL and direct Python calls. The architecture is designed to map cleanly onto a LangGraph StateGraph, enabling future migration without restructuring the business logic. Below is the conceptual LangGraph representation:

4.1 Node Responsibilities

INPUT NODE	• Receives: raw query string from Streamlit. • Produces: initialised GraphState = {query: q, context: '', response: '', status: 'PENDING'}. • Transitions: unconditionally to ROUTING EDGE.
ROUTING EDGE	• Evaluates: needs_human(state['query']). • If True: transitions state to HITL NODE. • If False: transitions state to PROCESSING NODE. • This is a conditional edge — the only branching point in the graph.
HITL NODE	• Receives: GraphState with status='PENDING'. • Executes: log_human_request(state['query']). • Mutates: state['response'] = escalation_msg; state['status'] = 'HITL'. • Transitions: to OUTPUT NODE (skips PROCESSING NODE).
PROCESSING NODE	• Executes retrieval: docs = retriever.invoke(state['query']). • Guard: if not docs → state['response'] = 'I don\'t know'; goto OUTPUT. • Builds context: '\n\n'.join(doc.page_content for doc in docs). • Constructs prompt: role + context + query + 'Answer:'. • Calls Groq API; on exception calls Falcon fallback. • Mutates: state['context'], state['response'], state['status'] = 'AI'.
OUTPUT NODE	• Reads: state['response'] and state['status']. • Returns: response string to app.py caller. • app.py checks ■ sentinel for UI rendering decision.

4.2 State Transitions

Event	Status	State Description
Query received	PENDING	query populated, others empty

HITL check = True	HITL	routed to human; LLM bypassed
HITL check = False	PROCESSING	retrieval initiated
Empty retrieval	AI (fallback)	response = 'I don\'t know.'
Groq success	AI	response = Groq answer
Groq failure	AI (degraded)	response = Falcon answer
Output returned	COMPLETE	response delivered to Streamlit

5. Conditional Logic & Intent Detection

Intent detection determines whether a query should be handled by AI or escalated to a human agent. The v1 implementation uses rule-based keyword matching — a deliberate choice prioritising zero false negatives on high-severity signals over NLU sophistication.

5.1 Two-Tier Detection Logic

```
def needs_human(query: str) -> bool:
    query = query.lower()
    # Tier 1: High-severity single-signal triggers
    if any(w in query for w in ['fraud','unauthorized','stolen','hack']):
        return True
    # Tier 2: Context-dependent combination triggers
    if 'card' in query and any(w in query for w in ['lost','block']):
        return True
    return False
```

Tier 1 keywords are domain-specific enough that any occurrence — regardless of surrounding context — is a reliable escalation signal. 'Fraud' in a banking context is never a casual query. **Tier 2** uses AND logic to avoid false positives: 'lost' alone might mean 'I lost my account number', but 'card' + 'lost' strongly implies card loss requiring human action.

5.2 Intent Detection Decision Matrix

Query	Match Path	Routing
'Report unauthorized transaction'	Tier 1: unauthorized	HITL
'I think there's fraud on my account'	Tier 1: fraud	HITL
'My card was stolen'	Tier 1: stolen	HITL
'Someone hacked my account'	Tier 1: hack	HITL
'Please block my card'	Tier 2: card+block	HITL
'I lost my debit card'	Tier 2: card+lost	HITL
'I lost my PIN'	No card → Tier 2 miss	AI
'Block my internet for a week'	No card → Tier 2 miss	AI
'How do I dispute a charge?'	No Tier 1/2 match	AI
'What is the minimum account balance?'	No Tier 1/2 match	AI

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop (HITL) is a design pattern that keeps a human decision-maker in the workflow for cases where automation alone is insufficient. In banking, HITL is not just a convenience — it is a compliance requirement. Fraud investigations and card-blocking requests require verified human judgement before action is taken.

Step	Action
Trigger	needs_human(query) returns True based on keyword match.
Log	log_human_request(query) appends raw query to human_requests.txt in append mode.
Bypass	The entire RAG pipeline (retrieval, prompt, LLM) is skipped — no AI response generated.
Notify User	A fixed escalation message with ■ prefix is returned to Streamlit.
Streamlit UI	app.py detects ■ sentinel → renders st.error() (red background) instead of st.success().
Human Action	Support agent reads human_requests.txt, identifies the query, and responds via separate channel.

6.2 Benefits and Limitations

- Zero AI hallucination risk on sensitive queries — the LLM never sees fraud/card-loss requests.
- Clear audit trail: human_requests.txt provides a timestampable log (future: add timestamps).
- Regulatory alignment: banking regulations (PCI-DSS, AML) require human oversight for certain event types.
- Simple and reliable: no ML model to fail — keyword matching is deterministic and explainable.
- Fast escalation: adds ~0 ms to response time for HITL queries (no API call).

Limitations:

- False Negative Risk: paraphrased or obfuscated fraud queries bypass keyword detection.
- No Response SLA: human_requests.txt is passive — no alerting or priority queue for agents.
- PII Exposure: raw query (potentially containing card numbers, names) written to plaintext file.
- Scalability: append-only flat file is not scalable for high-volume escalation queues.
- No Feedback Loop: escalated queries are never used to improve the keyword ruleset or train a better classifier.

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed	Dense vector search (HNSW) provides semantic accuracy but has higher compute overhead than sparse BM25 search. For a <50k-chunk knowledge base, HNSW at k=3 completes in < 10 ms on CPU — the accuracy benefit far outweighs the negligible latency cost. At 1M+ chunks, a hybrid sparse+dense approach (BM25 pre-filter → dense re-rank) would be appropriate.
Chunk Size vs Context Quality	Smaller chunks (< 200 chars) improve retrieval precision but may not contain enough context for the LLM to generate a complete answer. Larger chunks (> 1000 chars) provide richer context but degrade embedding quality (MiniLM truncates at 256 tokens). The 500-char choice sits at the empirically validated sweet spot for sentence-transformers on Q&A; tasks.
Cost vs Performance (LLM Choice)	GPT-4 Turbo would provide higher answer quality than llama-3.1-8b-instant but costs ~20× more per token. For banking FAQ queries (short, factual), 8B-parameter instruction-tuned models achieve near-parity quality at a fraction of the cost. Groq's LPU inference further compresses latency to sub-200ms — avoiding the speed tax that typically accompanies quality gains.
Keyword HITL vs NLU-based Routing	Keyword matching is brittle — it fails on paraphrases and non-English input. An NLU classifier (fine-tuned BERT on intent labels) would be more robust but introduces ML model failure modes, retraining overhead, and explainability challenges. For v1, the priority is zero false negatives on known high-risk patterns — keyword matching delivers this reliably.
Local Fallback vs No Fallback	Loading Falcon-RW-1B at startup consumes ~2–3 GB RAM and adds 30–60 seconds to cold-start time. The benefit is 100% availability even during Groq outages. Alternative: a queue-based retry with exponential backoff would avoid the memory overhead but introduce response delays during Groq degradation windows.

Stateless vs Stateful Conversation	The current system is fully stateless per query — each ask_question() call has no memory of prior turns. Streamlit session_state maintains chat history for display purposes only; it is not injected into the RAG pipeline. This simplifies the backend but means the LLM cannot resolve references like 'What about the previous answer?' Multi-turn memory (LangChain ConversationBufferMemory) is the v2 solution.

8. Testing Strategy

8.1 Testing Approach

Unit Testing — HITL Logic

- Test needs_human() with ≥ 20 positive samples (escalation expected) and ≥ 20 negative samples (AI expected).
- Verify Tier 1 keywords trigger independently of surrounding context.
- Verify Tier 2 requires both 'card' AND a modifier keyword.
- Test case-insensitivity: 'FRAUD', 'Fraud', 'fraud' all trigger.
- Test boundary: 'card' alone does NOT trigger.

Integration Testing — RAG Pipeline

- Submit 10 known questions from the PDF; verify answers contain expected keywords.
- Verify retriever returns exactly k=3 documents for all non-empty queries.
- Verify 'I don\'t know.' response for nonsensical/out-of-domain queries.
- Verify Groq response is stripped of whitespace.
- Test fallback path by mocking groq_client to raise Exception.

End-to-End Testing — Streamlit UI

- Verify escalation queries render with red (st.error) styling.
- Verify normal queries render with green (st.success) styling.
- Verify chat history persists across multiple messages in the same session.
- Verify 'Clear Chat' button resets session state to empty list.
- Test concurrent users by opening two browser tabs.

Performance Testing

- Measure ingestion time (PDF load → ChromaDB persist) for target PDF size.
- Measure p50/p95/p99 query latency for 100 sequential requests.
- Measure fallback LLM response time vs Groq primary.
- Measure cold-start time (Falcon model load) and warm-start time.

8.2 Sample Test Query Matrix

Query	Route	Expected Output	Status
'What documents are required for KYC?'	AI	Answer about KYC document list	PASS
'How do I check my account balance?'	AI	Balance checking instructions	PASS
'My card was stolen, what should I do?'	HITL	■ escalation message	PASS

'I want to block my credit card'	HITL	■ escalation message	PASS
'There is unauthorized activity on my account'	HITL	■ escalation message	PASS
'Someone hacked into my banking app'	HITL	■ escalation message	PASS
'What is the interest rate on savings accounts?'	AI	Interest rate information from PDF	PASS
'xyzzy quantum banana'	AI	'I don\'t know.'	PASS
'I lost my internet connection'	AI	AI response (no card keyword)	PASS
'block my friend from calling me'	AI	AI response (no card keyword)	PASS

9. Future Enhancements

Multi-Document Support

- Replace single-PDF loader with a document management layer: support PDF, DOCX, HTML, and web URLs.
- Implement metadata-aware retrieval: filter by document type, date, or topic before similarity search.
- Add document versioning: re-ingest updated policy documents without full DB rebuild using ChromaDB's upsert API.
- Introduce per-collection isolation: separate collections for loans, cards, KYC — enables domain-specific retrieval.

Feedback Loop & Continuous Improvement

- Thumbs up/down buttons in Streamlit UI capture per-answer quality signals.
- Low-rated answers trigger automated retrieval debugging: log query, chunks, and full prompt.
- Periodic fine-tuning of the embedding model on domain-specific Q&A; pairs using contrastive loss.
- A/B test different chunk sizes, k values, and prompt templates using logged feedback as ground truth.

Conversation Memory Integration

- Integrate LangChain ConversationBufferWindowMemory to maintain last N turns in context.
- Implement entity memory: extract and persist named entities (account numbers, dates) across turns.
- Design multi-turn RAG: include prior assistant answers in the retrieval query for follow-up resolution.
- Add session timeout and memory eviction to prevent context window overflow.

HITL v2 — NLU Classifier

- Replace keyword matching with a fine-tuned intent classifier (distilBERT or MiniLM on banking intents).
- Train on labelled dataset: HITL-class (fraud, loss, block) vs AI-class (FAQ, info, general).
- Add confidence thresholding: low-confidence AI predictions → HITL; high-confidence HITL predictions → immediate escalation.
- Build agent dashboard: FastAPI backend + React frontend replacing human_requests.txt flat file.

Production Deployment

- Containerise with Docker: multi-stage build separating model weights from application code.
- Deploy Streamlit via Kubernetes + HPA (Horizontal Pod Autoscaler) for traffic-based scaling.
- Migrate ChromaDB to Chroma Cloud or managed vector DB (Qdrant/Weaviate) for distributed retrieval.
- Implement CDN-cached responses for the top-100 most common queries (semantic deduplication).
- Add OAuth2 authentication, rate limiting (per-user), and end-to-end TLS.
- Structured logging (JSON) to ELK stack; Prometheus metrics for latency, throughput, error rate.

Advanced RAG Patterns

- HyDE (Hypothetical Document Embeddings): generate a hypothetical answer, embed it, retrieve similar chunks — improves recall for complex queries.
- Re-ranking: use a cross-encoder (ms-marco-MiniLM-L-6-v2) to re-rank top-k=10 chunks before selecting final top-3 for the prompt.
- Query expansion: use LLM to generate 3 alternative phrasings of the user query; retrieve for all; deduplicate results.
- Adaptive chunking: parse document structure (headings, tables, lists) and chunk at semantic boundaries rather than fixed character counts.

Summary Statement

The Banking RAG Customer Support System demonstrates a production-grade architecture for AI-powered customer service in regulated domains. By combining retrieval-augmented generation with deterministic human-in-the-loop routing, it achieves the dual objectives of automation efficiency and compliance safety — a model applicable across fintech, insurance, healthcare, and any domain where factual accuracy and risk governance are non-negotiable.